

P.PORTO

Deploy MEAN Stack Applications

Programação em Ambiente Web

Índice

Introdução

Deploy Node.js

Node.js e Angular

Introdução

Angular é uma framework *cliente-side*, ou seja, executa o seu código do lado do cliente

Isto implica que qualquer conteúdo dinâmico necessita de ser pedido a recursos na web

Comunicação com um servidor ou com base de dados tem de ser feita através de pedidos a cada um destes recursos na web

Introdução

Não é uma boa ideia uma aplicação em Angular ligar-se diretamente a uma base de dados:

- O código de ligação a base de dados ficará no cliente e implica ter a base de dados aberta a pedidos na web
- O cliente pode descobrir a palavra-passe da nossa ligação
- Riscos de segurança

Introdução

Questão:

Se o código da nossa aplicação é transferido para o cliente e executado como podemos adicionar por exemplo controlo de utilizadores?

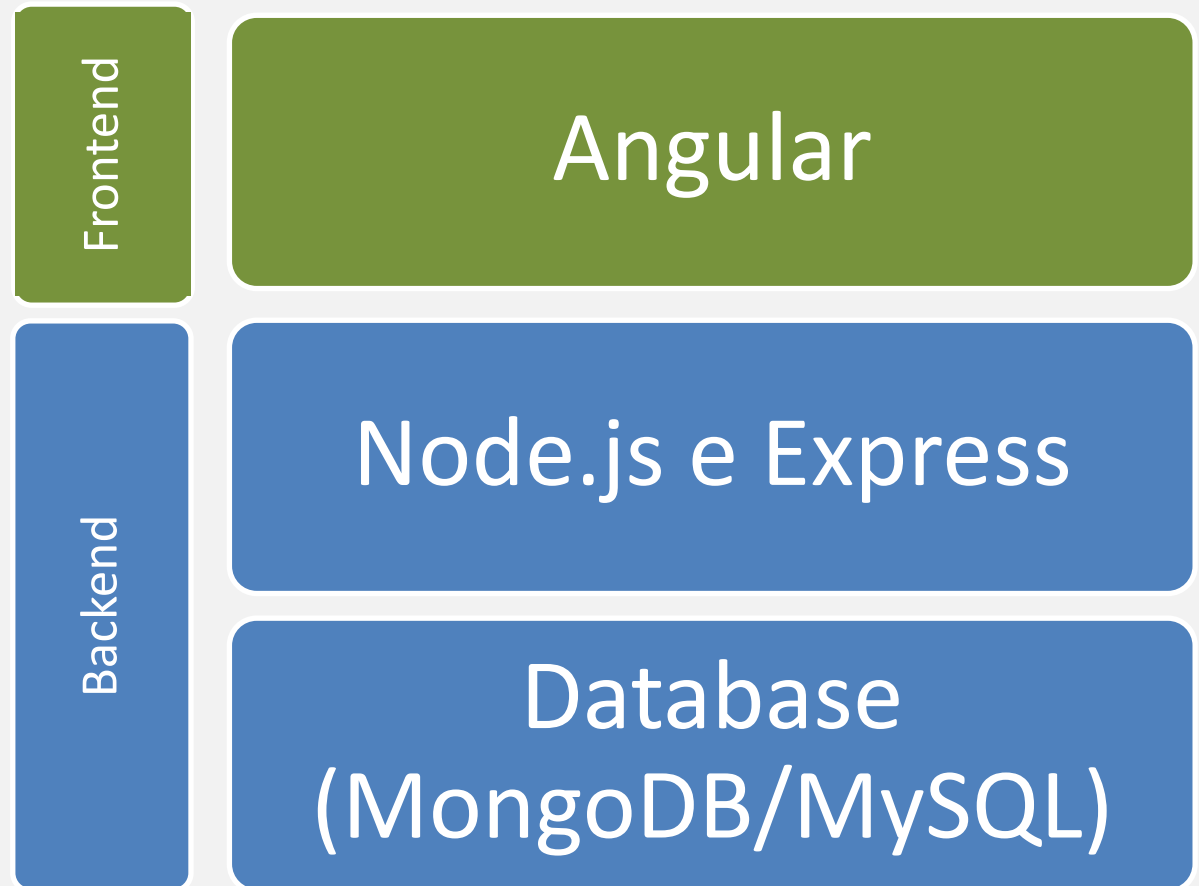
Introdução

Questão:

Como aceder a recursos ou APIs que necessitam de credenciais privadas e previno que um cliente “roube” as credenciais para outros fins?

Introdução

Aplicação
baseada na
MEAN Stack



Introdução

Para a correta resposta a cada uma destas questões devemos primeiro rever a estrutura de aplicações na MEAN Stack

Devemos separar conteúdo que está em backends do que está em frontend

Se o frontend necessita de conteúdo protegido deve pedir a recursos de backend e autenticar-se neles

Introdução

Os recursos protegidos para cada utilizador devem pedir as credenciais ao utilizador antes de efetuar os autenticação e pedidos aos recursos na web

Desta forma a aplicação em si não guardadas as credenciais diretamente no código da aplicação

Introdução

Questão:

E acesso a recursos e APIs externas para o qual o cliente não é suposto ter login?

Introdução

Devemos encapsular o acesso a estes serviços em aplicações backend e atuar como um broker deste serviço

Existem casos excepcionais com mecanismos de gestão de acesso diferentes onde pode não ser preciso o encapsulamento, mas deve ser avaliado caso a caso

Introdução

Para recursos livremente disponibilizados na web e que não necessitem de autenticação podemos efetuar pedidos diretamente a estes serviços

Deploy Node.js/ExpressJS Application

Deploy implica publicar a nossa aplicação num servidor público

Podemos escolher livremente um serviço de *hosting* que consiga dar resposta a estas características:

- AWS
- Google Cloud
- Azure
- (outros)

Deploy Node.js/ExpressJS Application

Para efectuar o deploy de uma aplicação Node.js/ExpressJS com uma base de dados mongoDB é necessário um servidor com pelo menos:

- ip e domínio publico
- servidor com nodeJS e ExpressJS
- base de dados mongoDB

Deploy Node.js/ExpressJS Application

Vamos explorar um exemplo baseado nos serviços heroku e mongoAtlas

Permitirá o deploy de uma aplicação com um plano *free* sujeito a algumas limitações:

- Número de horas que a aplicação pode correr por dia
- Capacidade do servidor
- Largura de banda para responder a pedidos

Deploy Node.js/ExpressJS Application

Heroku:

- Servidor com Node.JS e Express



Mongo Atlas:

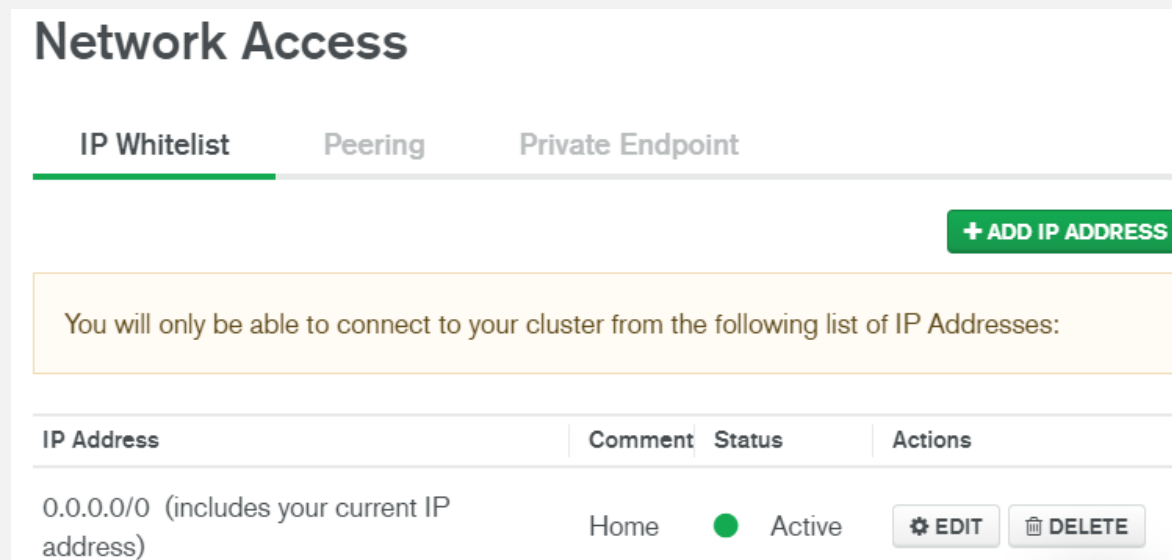
- Instância MongoDB pública que iremos usar a partir da nossa aplicação instalada na plataforma heroku



Deploy Node.js/ExpressJS Application

Mongo Atlas:

- Adicionar regra para permitir qualquer ip aceder à base de dados



The screenshot shows the 'Network Access' configuration page in MongoDB Atlas. The 'IP Whitelist' tab is selected, indicated by a green underline. A green button labeled '+ ADD IP ADDRESS' is in the top right. A yellow warning box states: 'You will only be able to connect to your cluster from the following list of IP Addresses:'. Below this is a table with columns: IP Address, Comment, Status, and Actions.

IP Address	Comment	Status	Actions
0.0.0.0/0 (includes your current IP address)	Home	● Active	<button>EDIT</button> <button>DELETE</button>

Deploy Node.js/ExpressJS Application

Mongo Atlas:

- Criar um utilizador da base de dados

ORGANIZATION 2066606 > PROJECT 0

Database Access

Database Users Custom Roles

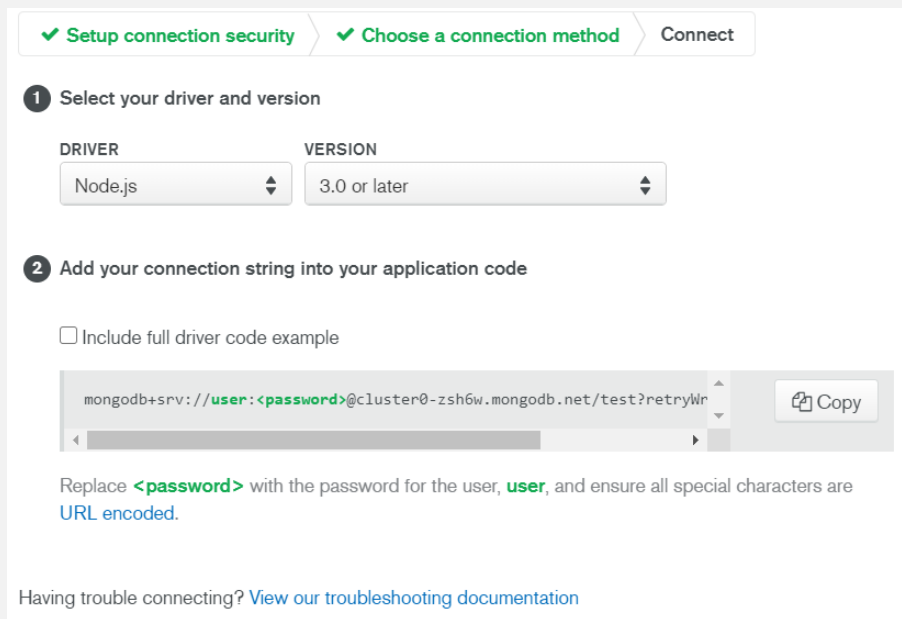
[+ ADD NEW DATABASE USER](#)

User Name	Authentication Method	MongoDB Roles	Actions
 user	SCRAM	readWriteAnyDatabase@admin	EDIT DELETE

Deploy Node.js/ExpressJS Application

Mongo Atlas:

- Retirar a url de conexão para incluir na nossa aplicação



The screenshot shows the MongoDB Atlas connection setup interface. At the top, there are three tabs: 'Setup connection security' (checked), 'Choose a connection method' (checked), and 'Connect'. Below the tabs, there are two main steps:

- 1 Select your driver and version**
 - DRIVER**: A dropdown menu showing 'Node.js'.
 - VERSION**: A dropdown menu showing '3.0 or later'.
- 2 Add your connection string into your application code**
 - ☐ Include full driver code example
 - A text area containing the connection string: `mongodb+srv://user:<password>@cluster0-zsh6w.mongodb.net/test?retryW`. A 'Copy' button is next to it.
 - Below the text area, there is a note: 'Replace <password> with the password for the user, user, and ensure all special characters are URL encoded.'

At the bottom, there is a link: 'Having trouble connecting? View our troubleshooting documentation'.

Deploy Node.js/ExpressJS Application

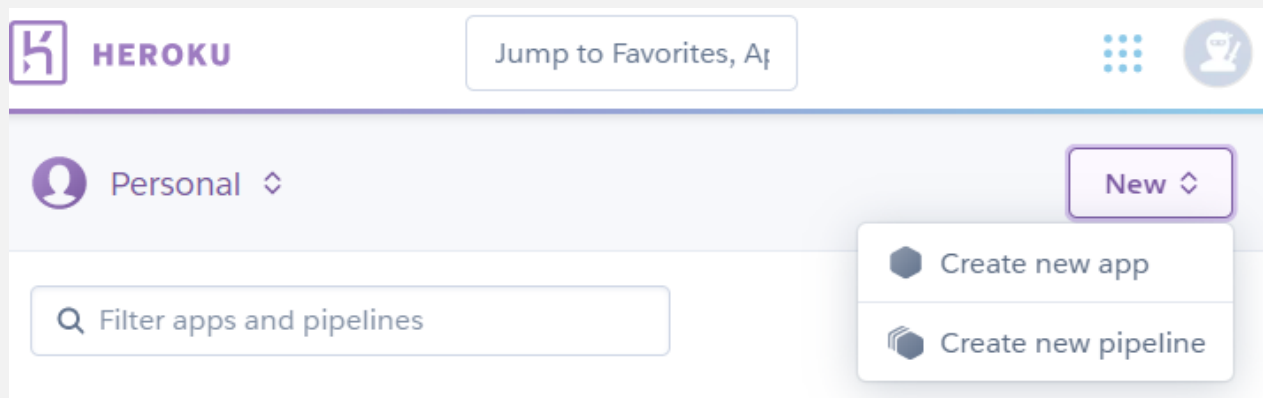
Necessitamos do mongo Atlas porque o serviço heroku não tem suporte *free* à base de dados Mongo Atlas

Também pode ser favorável, ter os dados em outro servidor

- permite arquiteturas avançadas para aplicações web
- permite replicar o servidor aplicacional à medida que o número de pedidos cresce
- (outros)

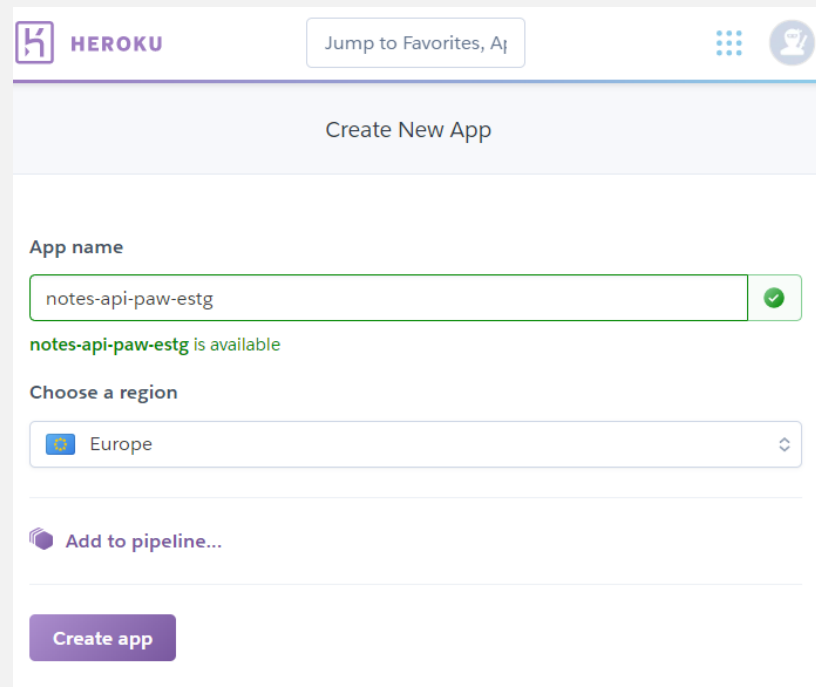
Deploy Node.js/ExpressJS Application

Depois do registo na aplicação heroku necessitamos de criar uma nova aplicação no serviço



Deploy Node.js/ExpressJS Application

Devemos escolher um nome público para a aplicação e a sua localização geográfica



The screenshot shows the Heroku 'Create New App' interface. At the top, there is a navigation bar with the Heroku logo, a 'Jump to Favorites, A' button, and a user profile icon. Below the navigation bar, the title 'Create New App' is centered. The main form area contains two sections: 'App name' and 'Choose a region'. The 'App name' section has a text input field with the value 'notes-api-paw-estg' and a green checkmark icon to its right. Below the input field, a message states 'notes-api-paw-estg is available'. The 'Choose a region' section has a dropdown menu with 'Europe' selected. At the bottom of the form, there is a link 'Add to pipeline...' and a purple 'Create app' button.

HEROKU Jump to Favorites, A

Create New App

App name

notes-api-paw-estg

notes-api-paw-estg is available

Choose a region

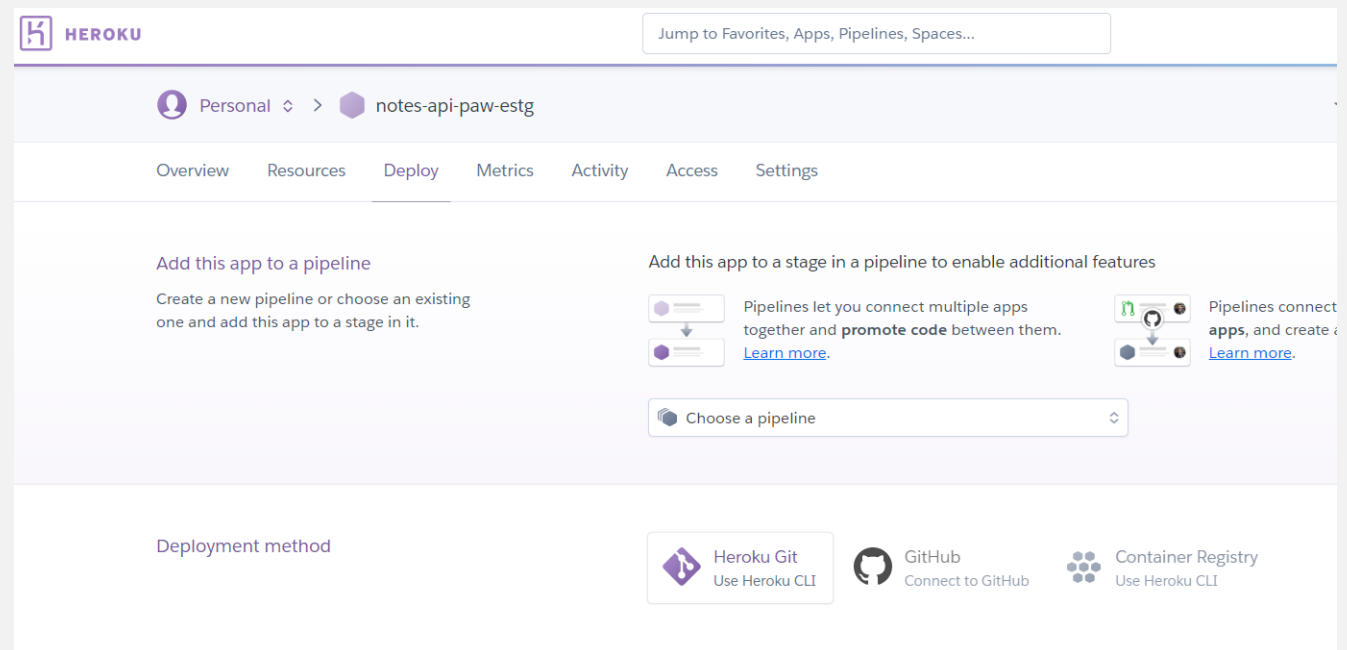
Europe

Add to pipeline...

Create app

Deploy Node.js/ExpressJS Application

Após a criação da aplicação, vamos usar a ferramenta heroku CLI para criar um remote para um repositório git dentro da plataforma heroku



Deploy Node.js/ExpressJS Application

A aplicação será instalada via upload do repositório git correspondente

Deploy using Heroku Git

Use git in the command line or a GUI tool to deploy this app.

Install the Heroku CLI

Download and install the [Heroku CLI](#).

If you haven't already, log in to your Heroku account and follow the prompts to create a new SSH public key.

```
$ heroku login
```

Create a new Git repository

Initialize a git repository in a new or existing directory

```
$ cd my-project/  
$ git init  
$ heroku git:remote -a notes-api-paw-estg
```

Deploy your application

Commit your code to the repository and deploy it to Heroku using Git.

```
$ git add .  
$ git commit -am "make it better"  
$ git push heroku master
```


Deploy Node.js/ExpressJS Application

Exemplo da lista de comandos para efectuar o upload do projeto para a plataforma heroku

- Criar um repositório git local
 - `git init`
 - `git add .`
 - `git commit -m 'message'`
- Inicializar o heroku-cli:
 - `heroku login`
 - `heroku git:remote -a [heroku-project-name]`
- Efectuar upload do projeto
 - `git push heroku master`

Deploy Node.js/ExpressJS Application

O heroku detecta que o nosso projeto é um projeto node.js/ExpressJS automaticamente

Corre por defeito os comandos:

- npm install
- (alguns comandos para compatibilizar a nossa aplicação com a plataforma heroku)
- npm start

Deploy Node.js/ExpressJS Application

Podemos agora usar a nossa api a partir de um ip publico com outras aplicações (ex:postman)

The screenshot shows the Postman interface for an 'Untitled Request'. The method is 'GET' and the URL is 'https://notes-api-paw-estg.herokuapp.com/api/v1/notes', which is highlighted with a red box. The 'Send' button is visible. Below the URL bar, the 'Query Params' section is empty. The 'Body' tab is selected, showing a JSON response in 'Pretty' format. The response status is '200 OK', the time is '287 ms', and the size is '369 B'. The JSON body contains the following data:

```
{  "_id": "5ed473f8ab29d609252811e2",  "title": "notes",  "description": "note description",  "__v": 0}
```

Deploy Node.js/ExpressJS Application

Podemos agora usar a nossa api a partir de um ip publico com outras aplicações (ex:postman)

The screenshot shows the Postman interface for an 'Untitled Request'. The method is 'GET' and the URL is 'https://notes-api-paw-estg.herokuapp.com/api/v1/notes', which is highlighted with a red box. The 'Send' button is visible. Below the URL bar, the 'Query Params' section is empty. The 'Body' tab is selected, showing a JSON response in 'Pretty' format. The response status is '200 OK', the time is '287 ms', and the size is '369 B'. The JSON body contains the following data:

```
{  "_id": "5ed473f8ab29d609252811e2",  "title": "notes",  "description": "note description",  "__v": 0}
```

Deploy Node.js/ExpressJS Application

Na nossa aplicação angular podemos agora usar a API publicada na plataforma heroku

Se juntarmos esta api com a aplicação angular deployed no serviço gitlab pages teremos a aplicação do cliente e servidor em serviços diferentes e a comunicar entre si

Node.js/Express e Angular

Podemos também incluir toda a aplicação (cliente e servidor) num só servidor escrito em node.js/express

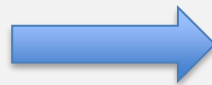
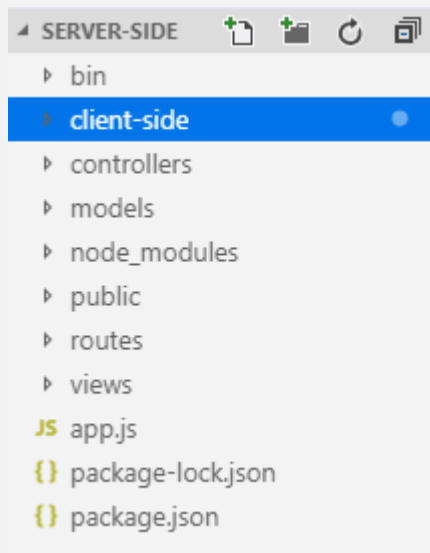
Neste caso iremos servir a nossa aplicação angular para dentro de um projeto de ExpressJS:

- apenas vamos criar uma mecanismo de cliente descarregar a aplicação em angular

Node.js e Angular

Podemos usar o runtime node.js e o middleware para servir páginas da aplicação em Angular

Incluímos o projeto Angular dentro de um projeto node.js/express



Aplicação angular!

Node.js e Angular

Precisamos de indicar ao middleware express para tratar a pasta dist/ da aplicação angular como uma pasta pública e com ficheiros estáticos

– Exemplo de um ficheiro app.js

```
var express = require('express');
var path = require('path');

var app = express();

app.use(express.static(path.join(__dirname, 'client-side', 'dist', 'client-side')));

app.use('/*', function (req, res) {
  console.log("sent");
  try{
    res.sendFile(path.join(__dirname, 'client-side', 'dist', 'client-side', 'index.html'));
  } catch(err){
    console.log(err);
  }
});

module.exports = app;
```


Node.js e Angular

Declaramos a pasta pública da aplica em angular

```
app.use(express.static(path.join(__dirname, 'client-side', 'dist', 'client-side')));
```

Enviamos o ficheiro index.html para todos os pedidos ao servidor para que a aplicação angular os trate

```
app.use('/*', function (req, res) {  
  console.log("sent");  
  try{  
    res.sendFile(path.join(__dirname, 'client-side', 'dist', 'client-side', 'index.html'));  
  } catch(err){  
    console.log(err);  
  }  
} );
```

Node.js/Express e Angular

Podemos na verdade também misturar a aplicação escrita em angular com uma aplicação node.js/express

Necessitamos de declarar routers específicos que sejam capazes de separar os pedidos http da aplicação em angular dos pedidos da aplicação node.js/express

Node.js/Express e Angular

```
var createError = require('http-errors');
var express = require('express');
var path = require('path');
var cookieParser = require('cookie-parser');
var logger = require('morgan');
var indexRouter = require('./routes/index');
var usersRouter = require('./routes/users');
var cors = require('cors');
var mongoose = require('mongoose');
mongoose.Promise = global.Promise;

mongoose.connect('mongodb://localhost:27020/express', {
  useNewUrlParser: true,
  useUnifiedTopology: true
})
.then(() => console.log('Connected to MongoDB'))
.catch((err) => console.error(err));

var app = express();

app.use(cors());
app.use(logger('dev'));
app.use(express.json());
app.use(express.urlencoded({ extended: false }));
app.use(cookieParser());
app.use(express.static(path.join(__dirname, 'client-side', 'dist', 'client-side')));

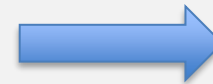
app.use('/api/v1/users', usersRouter);
app.use('/api/v1', indexRouter);

app.use('/*', function (req, res) {
  console.log("sent");
  try{
    res.sendFile(path.join(__dirname, 'client-side', 'dist', 'client-side', 'index.html'));
  } catch(err){
    console.log(err);
  }
});
```

Node.js/Express e Angular

Devemos editar o ficheiro package.json e editar o comando “start” para compilar a aplicação angular antes de iniciar o servidor com node.js

```
{
  "name": "server-side",
  "version": "0.0.0",
  "private": true,
  "scripts": {
    "start": "cd client-side && ng build && cd .. && node ./bin/www"
  },
  "dependencies": {
    "cookie-parser": "~1.4.3",
    "cors": "^2.8.5",
    "debug": "~2.6.9",
    "express": "~4.16.0",
    "http-errors": "~1.6.2",
    "jade": "~1.11.0",
    "mongoose": "^5.5.7",
    "moongoose": "0.0.5",
    "morgan": "~1.9.0"
  }
}
```



Alterar!

Referências

Heroku:

- <https://dashboard.heroku.com/>
- <https://devcenter.heroku.com/>

Mongo Atlas:

- <https://www.mongodb.com/cloud/atlas>
- <https://docs.mongodb.com/guides/>

P.PORTO

Deploy MEAN Stack Applications

Programação em Ambiente Web